

REFERENCE NOTES

Phoenix Architecture

Application · Endpoint · Router · LiveView routing
Contexts · Authentication · the full request flow
Counter · LiveDashboard · LiveView basics · Posts lab

Elixir Training Programme

Elixir · Phoenix framework architecture

Reference notes · Instructor-led · Elixir 1.20.4 on Erlang/OTP 29

PHOENIX FRAMEWORK · ARCHITECTURE REFERENCE

How a Phoenix application is put together

This guide follows a request through a Phoenix application, layer by layer. It starts with the **OTP application** that boots everything, then the **Endpoint** and **Router**, then LiveView routing, **contexts** with Ecto, authentication, and finally the full request flow from browser to database and back. Sections 11–13 build a **LiveView counter**, tour **LiveDashboard** and recap the **LiveView basics**; Section 14 is a hands-on lab on the class project **my_app**: a bug fix and two new features, step by step.

The layers, in order

- **Application** — starts everything (supervision tree)
- **Endpoint** — the front door and common plugs
- **Router** — routes, pipelines, auth protection
- **LiveView / Controller** — user interaction
- **Context** — business logic and database operations
- **Repo + Database** — persistence

What to look for

- Which module starts first, and why
- What each plug in the Endpoint does
- How **live_session** and **on_mount** protect pages
- Why LiveViews talk only to contexts
- Where each piece of authentication lives
- How events, assigns and timers drive a LiveView page

Contents

#	Section	What it covers
1	Introduction to Phoenix	What Phoenix is, its core features, getting started
2	Architecture of a Phoenix application	The layers and the directory structure
3	Which program initializes first?	The OTP application and its supervision tree
4	Endpoint	The front door: the plug pipeline every request passes
5	Router	Pipelines, scopes, routes, dispatch
6	LiveView routing	live routes, live_session, on_mount, the LiveView lifecycle
7	Contexts and database operations	Context API, schema and changeset, Repo
8	Authentication flow	Where auth lives in each layer; the login flow
9	Full request flow	Browser to database and back
10	Summary of layers	The six layers in order
11	A simple LiveView counter	mount, handle_event, render — step by step
12	Phoenix LiveDashboard	The built-in monitoring tool and its pages
13	Phoenix LiveView basics	Callbacks, the socket, bindings, HEEx, the mental model
14	Lab: the Posts page in my_app	Bug fix, Delete, a self-hiding success message

ABOUT THIS EDITION The source was checked against the **Phoenix 1.8.15**, **LiveView 1.2.12**, **LiveDashboard 0.8.7** and **Ecto 3.14.2** sources. Seven places where the code or the description does not match are flagged in amber **CORRECTION** boxes, each with the fix; four violet notes add precision (start-up order, Phoenix 1.8 names, the class project's names, LiveDashboard prerequisites). Everything else is as in the source. Section 14's code is copied from the class project after it was compiled and tested in the browser.

1

Introduction to Elixir Phoenix

the most popular web framework for Elixir

Phoenix is the most popular web framework for the **Elixir** programming language.

Aspect	Phoenix
Language	Elixir — built on the Erlang VM (BEAM)
Style	Similar to Ruby on Rails in structure (MVC, routers, controllers, views/templates, contexts), but designed for high concurrency, real-time features and fault tolerance
Key strengths	Extremely scalable and concurrent (Elixir/Erlang's actor model) · built-in real-time via Channels and LiveView · excellent performance and low latency · fault-tolerant by design

Core features

Feature	Description
Phoenix LiveView	Rich, real-time interactive UIs with mostly server-side Elixir (very little JavaScript)
Channels	WebSocket-based real-time communication
PubSub	Built-in publish-subscribe system
Ecto	Database wrapper and query language (usually used with PostgreSQL)
Router & Plugs	A powerful, composable request pipeline
Generators	<code>mix phx.gen.*</code> for scaffolding

Getting started

INSTALL PHOENIX, CREATE A PROJECT, START THE SERVER

```
# 1. install Elixir + Erlang, then the Phoenix project generator
mix archive.install hex phx_new

# 2. create a new project
mix phx.new my_app

# 3. start the server
cd my_app
mix phx.server
```

Resource	Where
Website	phoenixframework.org
Guides	hexdocs.pm/phoenix
LiveView docs	hexdocs.pm/phoenix_live_view

2

Architecture of a Phoenix application

a layered design on top of OTP

Phoenix follows a clean **layered architecture** that separates concerns, while leaning on Elixir's OTP strengths: supervision trees, processes and fault tolerance.

Layer	Location	Responsibility
Business / Domain	lib/my_app/	Contexts, schemas, business logic, database interaction (the "Model")
Web / Interface	lib/my_app_web/	HTTP handling, routing, controllers, LiveViews, components, templates (Controller + View)
Configuration	config/	Environment-specific settings
Assets	assets/	JS, CSS (esbuild + Tailwind by default)
Runtime resources	priv/	Migrations, static files, gettext, seeds
OTP application	lib/my_app/application.ex	The supervision tree that starts everything

DIRECTORY STRUCTURE · PHOENIX 1.7 / 1.8 (CONTEXT FILES CORRECTED)

```

my_app/
├── assets/           # Frontend source (JS, CSS, Tailwind)
├── config/           # config.exs, runtime.exs, dev/prod/test.exs
├── lib/
│   ├── my_app/       # Business domain
│   │   ├── application.ex # OTP Application + supervision tree
│   │   ├── repo.ex     # Ecto Repo
│   │   ├── mailer.ex
│   │   ├── accounts.ex  # Context: the public API (list_users, get_user, ...)
│   │   └── accounts/    # the context's schemas
│   │       └── user.ex   # Ecto Schema + changesets
│   ├── my_app.ex
│   ├── my_app_web/    # Web interface
│   │   ├── components/ # CoreComponents, Layouts, function components
│   │   ├── controllers/ # Traditional controllers + *_html.ex
│   │   ├── live/       # LiveViews
│   │   ├── endpoint.ex  # Entry point for all HTTP/WebSocket requests
│   │   ├── router.ex
│   │   ├── telemetry.ex
│   │   └── gettext.ex
│   └── my_app_web.ex
├── priv/
│   ├── repo/migrations/
│   ├── static/
│   └── gettext/
├── test/
└── mix.exs

```

CORRECTION 1 The source tree shows the context as **accounts/accounts.ex**. The generator writes the public API next to the folder, as **lib/my_app/accounts.ex**, and puts the schemas inside **lib/my_app/accounts/**. Section 7's own code comment (**# lib/my_app/accounts.ex**) already says so. Source: **Mix.Phoenix.Context** in Phoenix 1.8.15 (**file = dir <> ".ex"**).

3

Which program initializes first?

the OTP application module

Your application's first module is the **OTP Application module**: `lib/my_app/application.ex`, for example `MyApp.Application`. When you run `mix phx.server`, or start a release in production, the Erlang/Elixir VM starts your application by calling the `start/2` callback of this module. Everything else in the Phoenix app is started **from here**.

LIB/MY_APP/APPLICATION.EX

```
defmodule MyApp.Application do
  use Application

  @impl true
  def start(_type, _args) do
    children = [
      # 1. Telemetry (metrics & monitoring)
      MyAppWeb.Telemetry,

      # 2. Database connection pool (Ecto)
      MyApp.Repo,

      # 3. PubSub (for real-time messaging)
      {Phoenix.PubSub, name: MyApp.PubSub},

      # 4. The web server (Endpoint)
      MyAppWeb.Endpoint
    ]

    opts = [strategy: :one_for_one, name: MyApp.Supervisor]
    Supervisor.start_link(children, opts)
  end
end
```

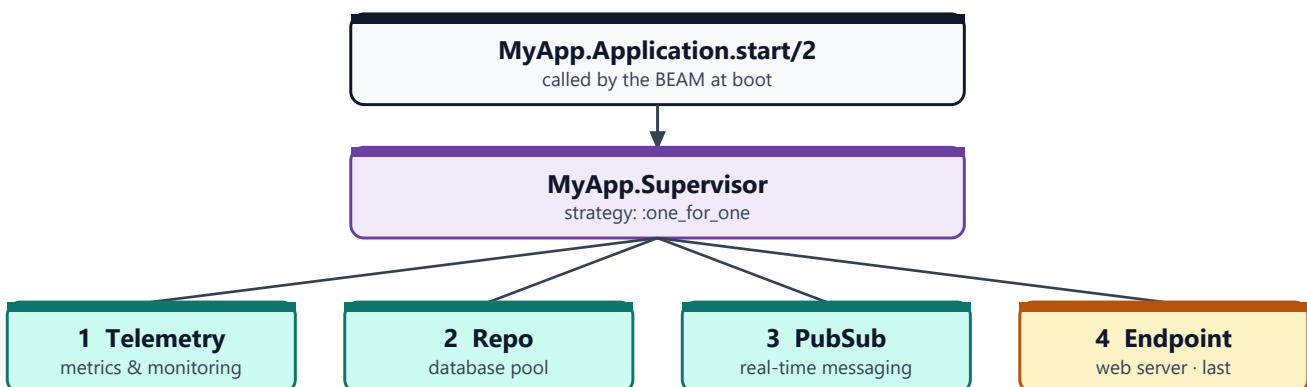


Figure 1. The supervision tree. Children start in **list order** (1 → 4) and stop in **reverse order**. With `:one_for_one`, a crashed child is restarted on its own.

- This is a standard **Elixir OTP Application**, and it defines a **supervision tree**.
- The children are started **in the order they appear** in the `children` list.
- If any child crashes, the supervisor can restart it (fault tolerance).
- When the application shuts down, the children are stopped in **reverse order**.

Startup sequence

#	Step
1	BEAM starts → calls MyApp.Application.start/2
2	The supervisor is created
3	Telemetry starts
4	Repo starts (connects to the database)
5	PubSub starts
6	Endpoint starts — the web server begins accepting HTTP requests

PRECISION Two details complete the picture. The VM knows which module to call from `mix.exs: def application, do: [mod: {MyApp.Application, []}, ...]`. And before your `start/2` runs, the BEAM has already started the applications your app **depends on** (`:logger`, `:telemetry`, `:phoenix`, `:ecto`, ...), in dependency order. `MyApp.Application` is the first of **your** modules to run.

4

Endpoint

`lib/my_app_web/endpoint.ex` · the entry point for every request

The Endpoint is the **entry point** for every HTTP and WebSocket request that comes into your Phoenix application.

- It is the first Phoenix module that touches an incoming request.
- It runs a series of **Plugs** (middleware) that prepare the request.
- At the end, it hands the request over to the **Router**.
- It also manages the web server (**Bandit** by default in modern Phoenix).

LIB/MY_APP_WEB/ENDPOINT.EX · TYPICAL STRUCTURE, AS IN THE SOURCE

```
defmodule MyAppWeb.Endpoint do
  use Phoenix.Endpoint, otp_app: :my_app

  # 1. Serve static files (CSS, JS, images, etc.)
  plug Plug.Static,
    at: "/",
    from: :my_app,
    gzip: false,
    only: MyAppWeb.static_paths()

  # 2. Code reloading (only in development)
  if code_reloading? do
    socket "/phoenix/live_reload/socket", Phoenix.LiveReloader.Socket
    plug Phoenix.LiveReloader
    plug Phoenix.CodeReloader
    plug Phoenix.Ecto.CheckRepoStatus, otp_app: :my_app
  end

  # 3. Request ID (useful for logging & tracing)
  plug Plug.RequestId

  # 4. Telemetry (metrics)
  plug Plug.Telemetry, event_prefix: [:phoenix, :endpoint]
```

```
# 5. Parse request body (JSON, form data, multipart, etc.)
plug Plug.Parsers,
  parsers: [:urlencoded, :multipart, :json],
  pass: ["*/*"],
  json_decoder: Phoenix.json_library()

# 6. Method override (support PUT/DELETE via forms)
plug Plug.MethodOverride

# 7. Head requests
plug Plug.Head

# 8. Session management
plug Plug.Session, @session_options

# 9. Finally → hand over to the Router
plug MyAppWeb.Router

end
```

CORRECTION 2 The code uses `@session_options` without defining it, and it has no `socket "/live"` line, yet that line is how LiveView's WebSocket reaches the app. A generated Phoenix 1.8 endpoint starts like this, before `Plug.Static` (from a project generated with `phx.new 1.8.14`):

THE MISSING LINES · TOP OF A GENERATED ENDPOINT.EX

```
@session_options [
  store: :cookie,
  key: "_my_app_key",
  signing_salt: "...",
  same_site: "Lax"
]

socket "/live", Phoenix.LiveView.Socket,
  websocket: [connect_info: [session: @session_options]],
  longpoll: [connect_info: [session: @session_options]]
```

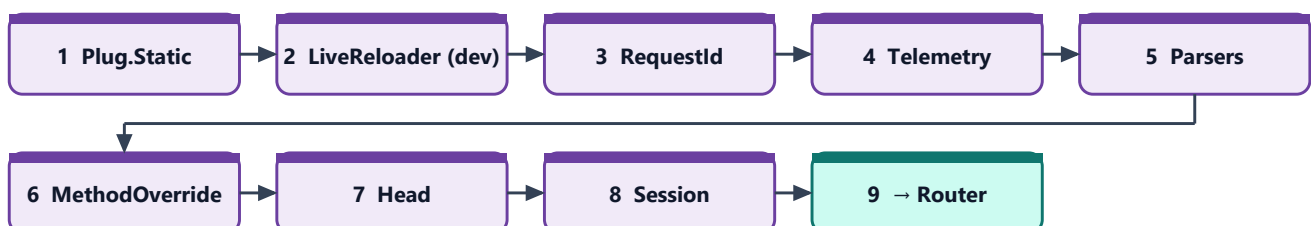


Figure 2. The plug pipeline every request passes through. **Plug.Static** answers CSS/JS/images straight away; **Parsers** turns the body into params; **Session** loads the session; then control passes to the **Router**.

Characteristic	Description
Single entry point	All requests (browser, API, LiveView) go through it
Plug pipeline	Highly composable — you can add your own plugs
WebSocket support	Also handles the LiveView and Channels sockets
Configuration	Settings come from <code>config/*.exs</code> (port, host, SSL, etc.)
Started by the Application	It is the last child started in <code>application.ex</code>

5

Router

lib/my_app_web/router.ex · decides where a request goes

After the **Endpoint** finishes its common plugs, it hands the request to the **Router**. The Router decides **where** the request should go.

- Match the HTTP method + path (**GET** /users, **POST** /posts, ...).
- Run specific groups of plugs called **pipelines**.
- Dispatch the request to a **Controller action** or a **LiveView**.
- Define **scopes** (to organize routes, authentication, API versions, ...).

LIB/MY_APP_WEB/ROUTER.EX · TYPICAL STRUCTURE

```
defmodule MyAppWeb.Router do
  use MyAppWeb, :router

  # =====
  # PIPELINES (groups of plugs)
  # =====
  pipeline :browser do
    plug :accepts, ["html"]
    plug :fetch_session
    plug :fetch_live_flash
    plug :put_root_layout, html: {MyAppWeb.Layouts, :root}
    plug :protect_from_forgery
    plug :put_secure_browser_headers
  end

  pipeline :api do
    plug :accepts, ["json"]
  end

  # =====
  # SCOPES + ROUTES
  # =====
  scope "/", MyAppWeb do
    pipe_through :browser

    get "/", PageController, :home
    get "/hello", HelloController, :index
    get "/hello/:messenger", HelloController, :show

    # LiveView routes
    live "/dashboard", DashboardLive
  end

  # API example
  scope "/api", MyAppWeb do
    pipe_through :api
    resources "/users", UserController, only: [:index, :show]
  end
end
```

Concept	Explanation
Pipeline	A named group of plugs. You can create several (:browser, :api, :authenticated, ...)
pipe_through	Applies one or more pipelines to a group of routes
Scope	Organizes routes under a common path prefix and module namespace
Route macros	get, post, put, patch, delete, resources, live, live_session, ...

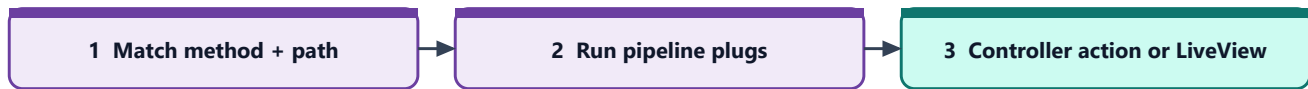


Figure 3. The Router's three jobs, in order.

CORRECTION 3 The source's Router flow lists **Channel (sockets)** as a third dispatch target. Channels do not pass through the Router. A channel socket is declared in the **Endpoint** (`socket "/socket", MyAppWeb.UserSocket`), just like LiveView's `socket "/live"`, and the socket module routes topics to channels. The Router dispatches only to controllers and LiveViews.

6

LiveView routing

live/3, live_session and on_mount

LiveView routes are defined in the **Router** with the `live/3` (or `live/4`) macro. They support **live navigation**, with no full page reload, while you stay inside the same **live_session**.

BASIC LIVEVIEW ROUTES

```

scope "/", MyAppWeb do
  pipe_through :browser

  live "/", HomeLive
  live "/users", UserLive.Index
  live "/users/:id", UserLive.Show
  live "/users/:id/edit", UserLive.Show, :edit
end
  
```

The optional last argument (`:edit`) is the **live action**: the same LiveView, told which mode to render in (`socket.assigns.live_action`).

live_session — the most important concept

live_session groups LiveViews that can navigate between each other **without a full page reload**.

THREE LIVE SESSIONS

```

scope "/", MyAppWeb do
  pipe_through :browser

  live_session :default do
    live "/", HomeLive
    live "/posts", PostLive.Index
    live "/posts/:id", PostLive.Show
  end

  live_session :authenticated, on_mount: {MyAppWeb.UserAuth, :ensure_authenticated} do
    live "/dashboard", DashboardLive
    live "/settings", SettingsLive
    live "/profile", ProfileLive
  end

  live_session :admin, on_mount: {MyAppWeb.UserAuth, :ensure_admin} do
    live "/admin", AdminLive.Dashboard
    live "/admin/users", AdminLive.Users
  end
end

```

Behavior	Explanation
Same live_session	Live navigation (<.link navigate> / push_navigate) stays on the same WebSocket — very fast
Different live_session	Always does a full page reload
on_mount	Runs shared logic (auth, assigns, ...) for every LiveView inside that session
Name	Must be unique (:default, :authenticated, :admin, ...)

CORRECTION 4 The source names **live_redirect** for live navigation. It is **deprecated** in LiveView ("Use <.link> instead"). Use **<.link navigate={...}>** in templates and **push_navigate/2** in code; the table above already says so.

on_mount hooks

LIB/MY_APP_WEB/USER_AUTH.EX

```

defmodule MyAppWeb.UserAuth do
  import Phoenix.LiveView
  import Phoenix.Component

  def on_mount(:ensure_authenticated, _params, session, socket) do
    socket = mount_current_user(socket, session)

    if socket.assigns.current_user do
      {:cont, socket}
    else
      {:halt, redirect(socket, to: ~p"/users/log_in")}
    end
  end
end

```

- **{:cont, socket}** lets the mount continue; **{:halt, socket}** stops it. Here it redirects to the login page.
- **mount_current_user/2** is a helper in the same module: it reads the session token and assigns the user.

- `~p"..."` is a verified route. The module needs `use MyAppWeb, :verified_routes` for it, which the generated `UserAuth` has.

PHOENIX 1.8 These are the **Phoenix 1.7** names. The **1.8** generator calls the hook `:require_authenticated`, assigns `current_scope` (with the user inside) instead of `current_user`, and uses `~p"/users/log-in"`. The mechanism is identical.

How LiveView routing works internally

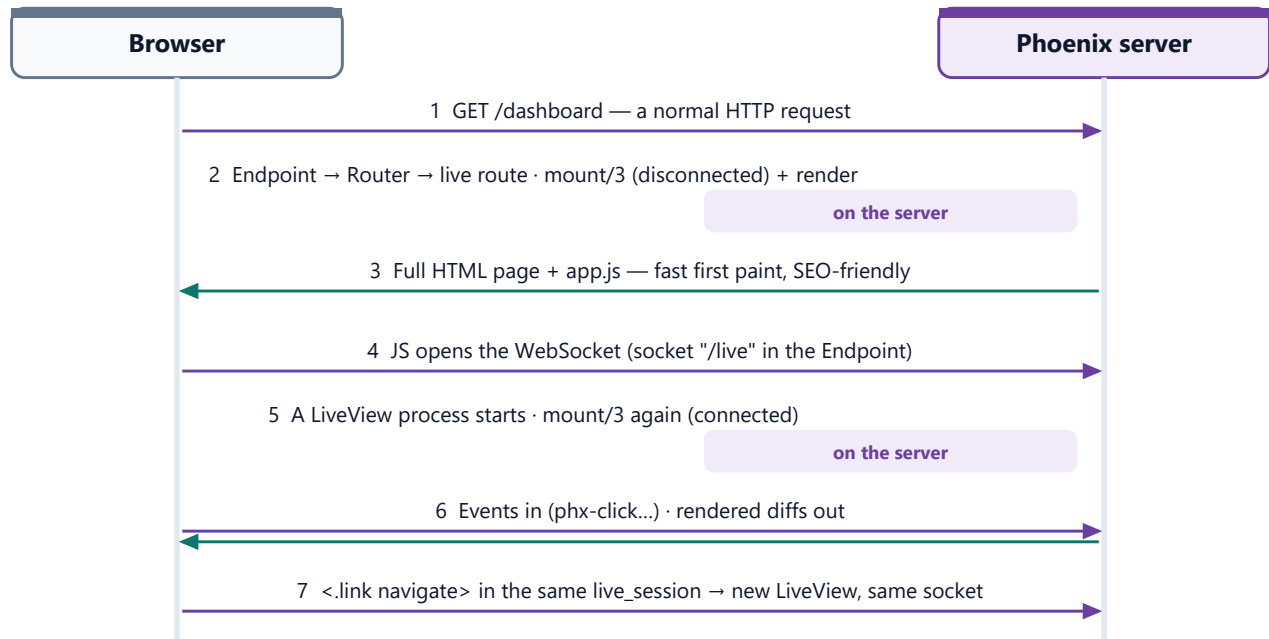


Figure 4. One LiveView page, from the first request to live navigation. The first render is plain HTML (the **disconnected** mount); the LiveView process exists only after the WebSocket connects.

7

Contexts (domain layer) and database operations

the heart of Phoenix's domain architecture

After the request reaches a **LiveView** (or a controller), the next important layer is the **Context**: a module that groups related business logic and database operations.

- It is the **public API** of your domain.
- LiveViews and controllers should **only** talk to contexts.
- They should **never** call **Repo** directly.

STRUCTURE

```

lib/my_app/
├── accounts.ex           ← Context: the public API (list_users, get_user, create_user...)
├── accounts/
│   └── user.ex          ← Ecto Schema + changesets
├── blog.ex
├── blog/
│   ├── post.ex
│   └── comment.ex
└── repo.ex
  
```

LIB/MY_APP/ACCOUNTS.EX · A TYPICAL CONTEXT

```
defmodule MyApp.Accounts do
  @moduledoc """
  The Accounts context.
  """

  import Ecto.Query, warn: false
  alias MyApp.Repo
  alias MyApp.Accounts.User

  def list_users do
    Repo.all(User)
  end

  def get_user!(id), do: Repo.get!(User, id)

  def get_user_by_email(email) do
    Repo.get_by(User, email: email)
  end

  def create_user(attrs \\ %{}) do
    %User{}
    |> User.changeset(attrs)
    |> Repo.insert()
  end

  def update_user(%User{} = user, attrs) do
    user
    |> User.changeset(attrs)
    |> Repo.update()
  end

  def delete_user(%User{} = user) do
    Repo.delete(user)
  end

  def change_user(%User{} = user, attrs \\ %{}) do
    User.changeset(user, attrs)
  end
end
```

LIB/MY_APP/ACCOUNTS/USER.EX · AS IN THE SOURCE

```
defmodule MyApp.Accounts.User do
  use Ecto.Schema
  import Ecto.Changeset

  schema "users" do
    field :email, :string
    field :name, :string
    field :password_hash, :string

    timestamps()
  end

  def changeset(user, attrs) do
    user
    |> cast(attrs, [:email, :name, :password])
    |> validate_required([:email, :name])
    |> validate_format(:email, ~r/@/)
    |> unique_constraint(:email)
    |> put_password_hash()
  end
end
```

CORRECTION 5 This changeset **cannot run**: it casts **:password**, but the schema has no such field. Measured on Ecto 3.14.2: **ArgumentError: unknown field :password given to cast**. The plain-text password is never stored, so it becomes a **virtual** field (as phx.gen.auth does). **put_password_hash/1** is not an Ecto function: you write it, e.g. with Bcrypt or Argon2. And **unique_constraint(:email)** only reports a duplicate if the migration creates a **unique index** on **email**.

THE SCHEMA, CORRECTED (CHECKED: THE CHANGESET IS VALID WITH THE VIRTUAL FIELD)

```
schema "users" do
  field :email, :string
  field :name, :string
  field :password, :string, virtual: true, redact: true # cast, validated, never stored
  field :password_hash, :string

  timestamps()
end

defp put_password_hash(%Ecto.Changeset{valid?: true, changes: %{password: pw}} = cs),
  do: change(cs, password_hash: Bcrypt.hash_pwd_salt(pw)) # {:bcrypt_elixir, "~> 3.0"}

defp put_password_hash(changeset), do: changeset
```

Complete flow: LiveView → Context → Database

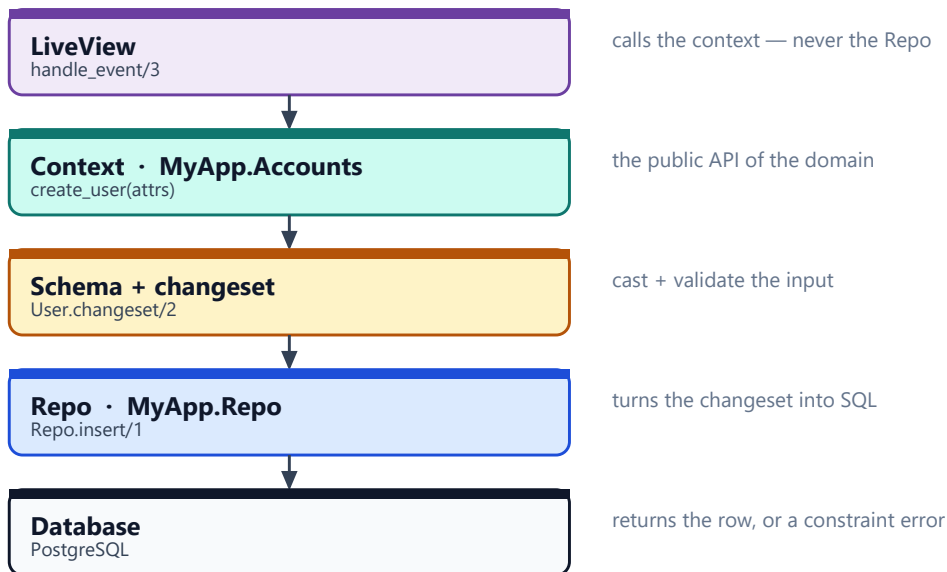


Figure 5. Each layer calls only the one below it. The result travels back up as `{:ok, user}` or `{:error, changeset}`.

EXAMPLE: CREATE A USER FROM A LIVEVIEW

```
def handle_event("save", %{"user" => user_params}, socket) do
  case Accounts.create_user(user_params) do
    {:ok, user} ->
      {:noreply,
       socket
       |> put_flash(:info, "User created successfully")
       |> push_navigate(to: ~p"/users/#{user}")}

    {:error, %Ecto.Changeset{} = changeset} ->
      {:noreply, assign(socket, form: to_form(changeset))}
  end
end
```

Important rules

#	Rule
1	LiveView / Controller → Context only — never call Repo from a LiveView or controller
2	A context returns <code>{:ok, struct}</code> on success and <code>{:error, changeset}</code> on validation failure
3	Changesets handle validation and data transformation

8

Authentication flow

not a separate layer — spread cleanly across the existing ones

Layer	What happens here related to auth
Context (Accounts)	Core business logic: register, login, logout, password hashing, tokens, sessions
Router	Pipelines + <code>live_session</code> + <code>on_mount</code> hooks to protect routes
Endpoint	The session plug (already there)
LiveView / Controller	Calls the context and reacts to success or failure
Plugs / <code>on_mount</code>	Check whether the user is authenticated

The structure `phx.gen.auth` creates

MIX PHX.GEN.AUTH ACCOUNTS USER USERS · AS IN THE SOURCE

```

lib/my_app/
├─ accounts/
│  ├─ accounts.ex      ← Main Context (public API)
│  ├─ user.ex          ← Schema
│  ├─ user_token.ex    ← For session tokens & password reset
│  └─ user_notifier.ex ← Emails
└─ ...

lib/my_app_web/
├─ user_auth.ex        ← on_mount hooks + plugs
├─ user_session_controller.ex
├─ live/
│  ├─ user_login_live.ex
│  ├─ user_registration_live.ex
│  └─ ...
└─ ...

```

As in Section 2, the context itself is `lib/my_app/accounts.ex`. In Phoenix 1.8 the LiveViews are grouped as `live/user_live/login.ex` (`UserLive.Login`), `registration.ex`, `settings.ex` and `confirmation.ex`, and the generator adds `accounts/scope.ex`.

Complete authentication flow

#	Step
1	The user tries to open a protected page: Browser → Endpoint → Router
2	The Router's <code>live_session</code> checks authentication with an <code>on_mount</code> hook
3	<code>on_mount</code> runs before the LiveView mounts: <code>{:cont, socket}</code> or <code>{:halt, redirect(...)}</code>
4	Login: the form's credentials are checked by the Accounts context; a session token is created and put in the session; the browser is redirected

STEP 2 · THE ROUTER PROTECTS A GROUP OF LIVEVIEWS

```

live_session :authenticated,
  on_mount: [{MyAppWeb.UserAuth, :ensure_authenticated}] do
    live "/dashboard", DashboardLive
    live "/settings", SettingsLive
  end

```

STEP 3 · ON_MOUNT RUNS BEFORE THE LIVEVIEW MOUNTS

```
def on_mount(:ensure_authenticated, _params, session, socket) do
  socket = mount_current_user(socket, session)

  if socket.assigns.current_user do
    {:cont, socket}      # Allow access
  else
    {:halt, redirect(socket, to: ~p"/users/log_in")}
  end
end
```

Step 4 · the login process

CORRECTION 6 The source shows the login LiveView calling `UserAuth.log_in_user(socket, user)`. That cannot work: logging in **writes the session cookie**, and only a normal HTTP response can set a cookie; a LiveView talks over the WebSocket. In the generated code (phx.gen.auth 1.8.15), the login LiveView validates the form and then submits it as a real **POST** (**phx-trigger-action**). `UserSessionController.create/2` checks the password through the context and calls `UserAuth.log_in_user(conn, user, params)`, which takes a **conn**.

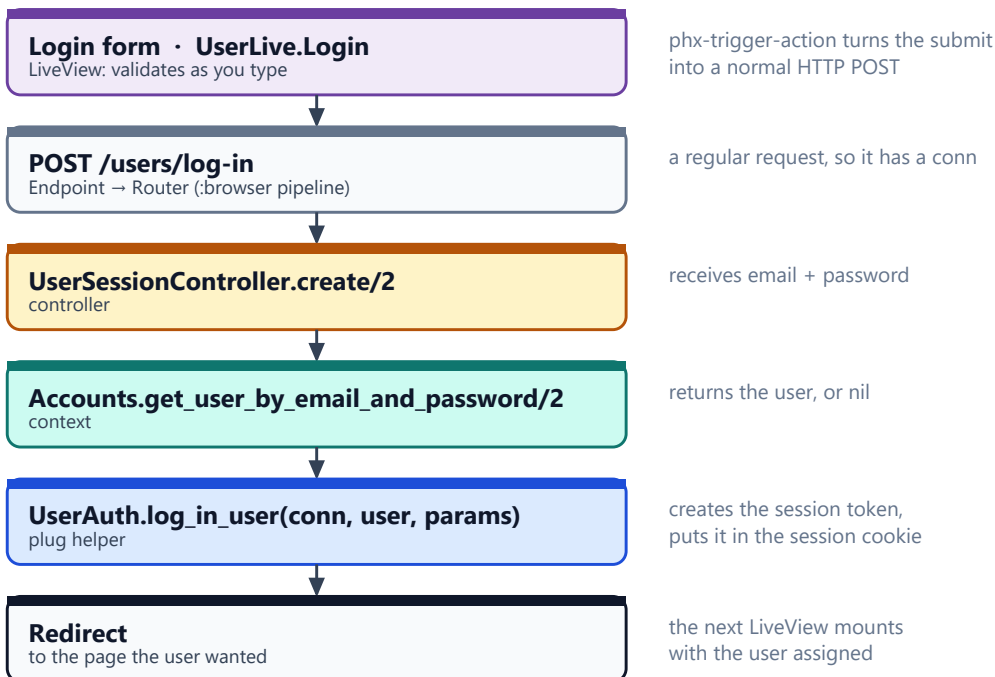


Figure 6. The login flow as `phx.gen.auth` generates it: the form lives in a LiveView, but the login itself happens in a controller.

Best-practice placement

Action	Where it should go
Register / login / logout logic	Accounts context
Password hashing and validation	User schema + changeset
Session token management	Accounts context
Protecting routes	Router (live_session + on_mount)
Checking the current user	UserAuth module (on_mount / plugs)
Login form UI	LiveView (UserLive.Login; UserLoginLive in 1.7)
Writing the session cookie	UserController via UserAuth.log_in_user(conn, ...)

9

Full request flow

the complete picture, browser to database and back

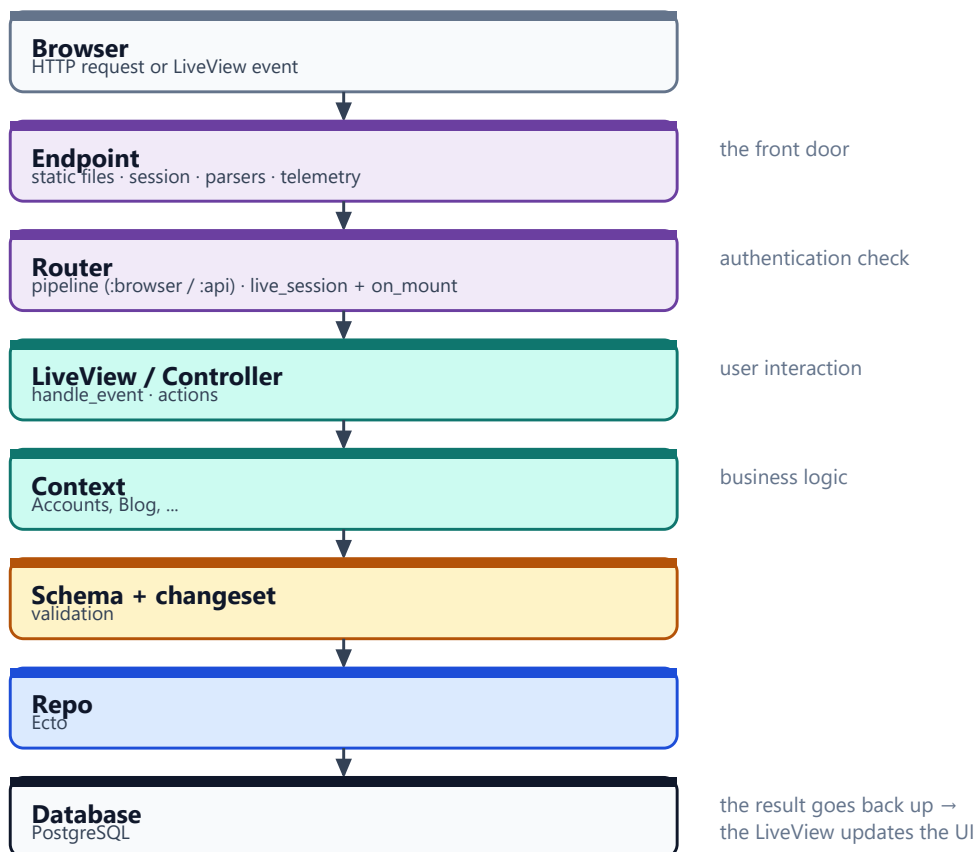


Figure 7. One request through every layer. On the way back, a LiveView re-renders and sends only the changed parts (a diff) to the browser.

10

Summary of layers in order

six layers, each with one job

#	Layer	Its job
1	Application	Starts everything (the OTP supervision tree)
2	Endpoint	Front door + common plugs
3	Router	Decision maker (routes + auth protection)
4	LiveView / Controller	Handles user interaction
5	Context	Business logic + database operations
6	Repo + Database	Data persistence

TAKEAWAY Read the list top-down for **start-up** (what boots what) and as a request path for **runtime** (what calls what). Each layer talks only to its neighbour, which is what keeps a Phoenix application easy to change.

11

A simple LiveView counter

step by step: the module, the route, running it

A counter is the smallest useful LiveView: one piece of state (**count**), three events and a template. It shows the whole LiveView cycle in about sixty lines.

IN THE CLASS PROJECT The source builds the counter in a project called **phoenix_live_demo**, so its paths and modules read **lib/phoenix_live_demo_web/...** and **PhoenixLiveDemoWeb**. The class project **my_app** has the identical counter at **lib/my_app_web/live/counter_live.ex** (**MyAppWeb.CounterLive**), routed in **lib/my_app_web/router.ex** and served at **http://localhost:4000/counter**.

Step 1 · Create the LiveView module

LIB/PHOENIX_LIVE_DEMO_WEB/LIVE/COUNTER_LIVE.EX

```
defmodule PhoenixLiveDemoWeb.CounterLive do
  use PhoenixLiveDemoWeb, :live_view

  @impl true
  def mount(_params, _session, socket) do
    {:ok, assign(socket, count: 0)}
  end

  @impl true
  def handle_event("increment", _params, socket) do
    {:noreply, update(socket, :count, &(&1 + 1))}
  end

  @impl true
  def handle_event("decrement", _params, socket) do
    {:noreply, update(socket, :count, &(&1 - 1))}
  end

  @impl true
```

```

def handle_event("reset", _params, socket) do
  {:noreply, assign(socket, count: 0)}
end

@impl true
def render(assigns) do
  ~H"""
  <div class="min-h-screen bg-gradient-to-br from-indigo-50 to-purple-100
    flex items-center justify-center p-6">
    <div class="bg-white rounded-2xl shadow-xl p-10 max-w-md w-full text-center">
      <h1 class="text-3xl font-bold text-gray-800 mb-2">LiveView Counter</h1>
      <p class="text-gray-500 mb-8">A simple real-time counter example</p>

      <div class="text-7xl font-bold text-indigo-600 mb-10 tabular-nums">
        {@count}
      </div>

      <div class="flex justify-center gap-4">
        <button phx-click="decrement"
          class="bg-red-500 hover:bg-red-600 text-white font-semibold py-3 px-6 rounded-xl">
          -
        </button>

        <button phx-click="reset"
          class="bg-gray-500 hover:bg-gray-600 text-white font-semibold py-3 px-6 rounded-xl">
          Reset
        </button>

        <button phx-click="increment"
          class="bg-green-500 hover:bg-green-600 text-white font-semibold py-3 px-6 rounded-xl">
          +
        </button>
      </div>

      <p class="mt-8 text-sm text-gray-400">
        This counter updates in real-time using Phoenix LiveView
      </p>
    </div>
  </div>
  """
end
end

```

The buttons in the source also carry **transition shadow-md**; the classes are shortened here to fit the page.

Part	What it does
mount/3	Runs when the LiveView starts. Sets the initial count to 0
handle_event("increment")	Increases the count by 1
handle_event("decrement")	Decreases the count by 1
handle_event("reset")	Sets the count back to 0
render/1	Returns the HTML (a HEEx template)
phx-click="..."	Sends the event to the server when the button is clicked

Step 2 · Add the route

LIB/PHOENIX_LIVE_DEMO_WEB/ROUTER.EX

```
scope "/", PhoenixLiveDemoWeb do
  pipe_through :browser

  get "/", PageController, :home

  live "/counter", CounterLive      # ← Added this line
end
```

This makes the LiveView available at **http://localhost:4000/counter**.

Step 3 · Run it

IN A TERMINAL

```
cd phoenix_live_demo
mix deps.get
mix ecto.create # optional for this example
mix phx.server
```

Then open **http://localhost:4000/counter**.

How the counter works

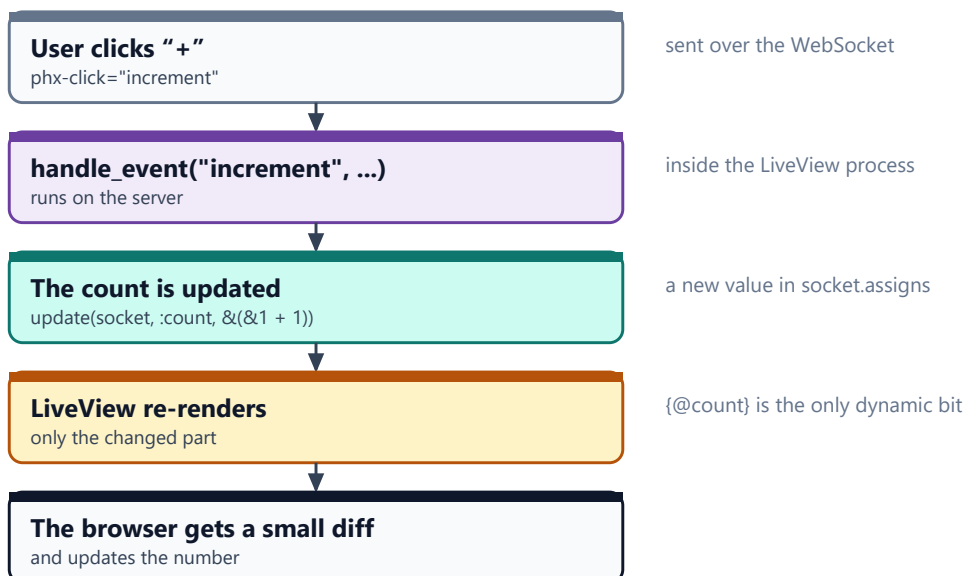


Figure 8. One click, one round trip. The HTML around the number is never sent again — only the new value of **@count** travels back.

Optional · link to the counter from the home page

Add a link to the home page template,

lib/phoenix_live_demo_web/controllers/page_html/home.html.heex:

HOME.HTML.HEEX

```
<a
  href="/counter"
  class="inline-flex items-center gap-2 rounded-xl bg-indigo-600 px-6 py-3 text-sm
    font-semibold text-white shadow-md transition hover:bg-indigo-700"
>
  Try the LiveView Counter →
</a>
```

In HEEx you can also write `<.link href={~p"/counter"}>`: the `~p` sigil checks the path against the router at compile time, so a typo becomes a compiler warning.

12

Phoenix LiveDashboard

real-time monitoring and debugging, built on LiveView

LiveDashboard is a real-time monitoring and debugging tool built into Phoenix. It is powered by **LiveView**, so it updates live without page refreshes. A project created with **mix phx.new** has it installed and configured.

How to access it

By default it is only available in development, at **http://localhost:4000/dev/dashboard**. The route is in **router.ex**:

ROUTER.EX · DEV-ONLY ROUTES

```
if Application.compile_env(:phoenix_live_demo, :dev_routes) do
  import Phoenix.LiveDashboard.Router

  scope "/dev" do
    pipe_through :browser
    live_dashboard "/dashboard", metrics: PhoenixLiveDemoWeb.Telemetry
  end
end
```

:dev_routes is set to **true** only in **config/dev.exs**, so this block is not compiled into a production build.

Main features and pages

Page	What you can see / do
Home	System overview: Elixir/OTP version, uptime, process count, memory, etc.
OS Data	CPU, memory and disk usage of the operating system
Metrics	Real-time charts of Telemetry metrics (request duration, memory, etc.)
Request Logger	See all logs related to a specific HTTP request
Applications	List of running applications and their supervision trees
Processes	Search and inspect any process on the node
Ports	View open ports (I/O)
Sockets	View open sockets (TCP/UDP)
ETS	Inspect ETS tables (in-memory storage)
Ecto Stats	Database statistics (if the Repo is running)

PRECISION LiveDashboard 0.8.7 has one more page, **Memory Allocators**. Two pages need extras, and their menu links are greyed out without them: **OS Data** needs the `:os_mon` application (add it to `extra_applications` in `mix.exs`; `my_app` lists only `:logger` and `:runtime_tools`), and **Ecto Stats** needs a running Repo **and** the `ecto_psql_extras` dependency.

Why LiveDashboard is useful

Use	What it gives you
Development	See instantly how your app behaves · debug slow requests · inspect processes and memory
Debugging	Click a request and see all its logs · find memory leaks or runaway processes
Performance monitoring	Real-time charts of request latency, memory, reductions, ...
Multi-node support	If several nodes are connected, switch between them

Metrics — the most powerful part

LiveDashboard draws real-time charts from **Telemetry** events. The project already has a Telemetry module that lists the metrics to chart:

LIB/PHOENIX_LIVE_DEMO_WEB/TELEMETRY.EX · EXCERPT

```
defmodule PhoenixLiveDemoWeb.Telemetry do
  use Supervisor
  import Telemetry.Metrics

  def metrics do
    [
      # Phoenix Metrics
      summary("phoenix.endpoint.start.system_time", unit: {:native, :millisecond}),
      summary("phoenix.endpoint.stop.duration", unit: {:native, :millisecond}),
      summary("phoenix.router_dispatch.start.system_time", unit: {:native, :millisecond}),
      summary("phoenix.router_dispatch.exception.duration", unit: {:native, :millisecond}),
      summary("phoenix.router_dispatch.stop.duration", unit: {:native, :millisecond}),
      summary("phoenix.socket_connected.duration", unit: {:native, :millisecond}),
      # ... more metrics
    ]
  end
end
```

These metrics appear as live charts under the **Metrics** tab.

SECURITY LiveDashboard can show process state, memory and more, so by default it is enabled only in **development**. In production, either disable it completely or protect it with authentication (Basic Auth or your own auth system).

Quick tips

- Open **/dev/dashboard** while you use the counter and watch the metrics update in real time.
- Go to **Processes** and search for **CounterLive** to find your LiveView process. This works because LiveView records **{MyAppWeb.CounterLive, :mount, 3}** as the process's initial call, and that is what the search matches.
- Use **Request Logger** to debug one specific request.

13

Phoenix LiveView basics

server-side state, small diffs over a WebSocket

LiveView lets you build rich, interactive web pages using almost only Elixir. The server keeps the state and sends small HTML diffs to the browser over a WebSocket.

1 · Core idea

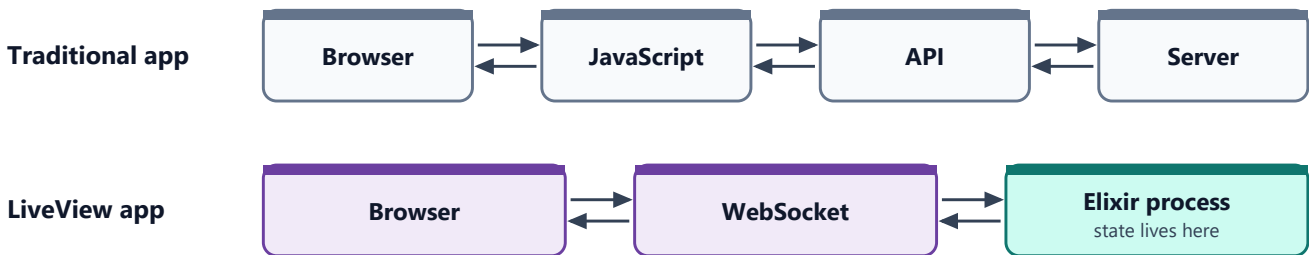


Figure 9. A traditional app splits the work between browser JavaScript and an API. In a LiveView app the browser talks to one Elixir process over a WebSocket.

- The **server owns the state** and **renders the HTML**.
- Only the **changed parts** are sent to the browser.
- You write very little (or zero) JavaScript.

2 · Basic LiveView structure

EVERY LIVEVIEW HAS THESE MAIN PARTS

```
defmodule MyAppWeb.CounterLive do
  use MyAppWeb, :live_view

  # 1. Initialize state
  def mount(_params, _session, socket) do
    {:ok, assign(socket, count: 0)}
  end

  # 2. Handle user events
  def handle_event("increment", _params, socket) do
    {:noreply, update(socket, :count, &(&1 + 1))}
  end

  # 3. Render HTML
  def render(assigns) do
    ~H"""
    <h1>Count: {@count}</h1>
    <button phx-click="increment">+</button>
    """
  end
end
```

3 · Important callbacks

Callback	When it runs	Purpose
mount/3	When the LiveView starts	Set the initial state
handle_event/3	When the user clicks / submits a form	React to user actions
handle_params/3	When the URL changes (live navigation)	React to URL parameters
handle_info/2	When a message arrives from a process	Handle PubSub / timers
render/1	After any state change	Generate the HTML

Section 14 uses **handle_info/2** for a timer: the LiveView sends itself a message three seconds later to hide a banner.

4 · The socket — your state container

READING AND CHANGING ASSIGNS

```
socket.assigns.count      # read a value
assign(socket, count: 5)  # set a value
update(socket, :count, &(&1 + 1)) # update a value
```

- Put only the data you need in **assigns**.
- Changing assigns automatically triggers a re-render.
- Keep the state small and clean.

5 · Common bindings (events)

Binding	Example	Description
phx-click	phx-click="increment"	Click event
phx-submit	phx-submit="save"	Form submit
phx-change	phx-change="validate"	Form field change
phx-value-*	phx-value-id={@id}	Pass extra data with the event
phx-disable-with	phx-disable-with="Saving..."	Disable the button while waiting

6 · Simple example recap (counter)

THE COUNTER, IN ITS SHORTEST FORM

```
def mount(_, _, socket) do
  {:ok, assign(socket, count: 0)}
end

def handle_event("inc", _, socket) do
  {:noreply, update(socket, :count, &(&1 + 1))}
end

def handle_event("dec", _, socket) do
  {:noreply, update(socket, :count, &(&1 - 1))}
end

def render(assigns) do
  ~H"""
  <h1>{@count}</h1>
  <button phx-click="dec">-</button>
  <button phx-click="inc">+</button>
  """
end
```

7 · Routing a LiveView

ROUTER.EX

```
live "/counter", CounterLive

# or with a live action
live "/users/:id", UserLive, :show
```

8 · HEEx template basics

HEEX

```
<!-- Comments --%>

<h1>{@title}</h1>                                <!-- Safe interpolation --%>

<div :if={@show}>Visible</div>                    <!-- Conditional --%>

<ul>
  <li :for={item <- @items}>{item.name}</li>
</ul>

<.link navigate={~p"/other"}>Go</.link>
```

9 · Mental model

#	As in the source
1	User opens the page → LiveView process starts (mount)
2	HTML is sent to the browser
3	WebSocket connection is established
4	User interacts → event is sent to the server
5	Server updates state → re-renders → sends only the diff
6	Browser updates the DOM

CORRECTION 7 Steps 1–3 are in the wrong order. The first HTML comes from a **disconnected** mount that runs inside the ordinary HTTP request; no LiveView process exists yet. The process starts only **after** the WebSocket connects, and **mount/3** runs a **second** time there (**connected?(socket)** is now **true**). Figure 4 in Section 6 shows the same sequence.

#	Corrected
1	The browser requests the page; Endpoint → Router finds the live route
2	mount/3 + render/1 run once inside the HTTP request (disconnected) → full HTML is sent
3	The browser loads app.js, which opens the WebSocket
4	A LiveView process starts; mount/3 runs again (connected)
5	User interacts → the event goes to that process → handle_event/3
6	The server updates state → re-renders → sends only the diff → the browser patches the DOM

14

Lab: the Posts page in my_app

a bug fix and two features, step by step

The class project **my_app** has a **Posts** page: a form to add or edit a post, and a table that lists the posts. There is no database — the posts live in memory, in an **Agent**. This lab fixes a bug in **Add Post**, adds a **Delete** button, and makes the green success message hide itself after three seconds. Every change was compiled with **mix compile --warnings-as-errors** and tested in the browser.

Item	Value
Project folder	D:\trainings\cts\session\webapps\my_app
Versions	Phoenix 1.8.15 · LiveView 1.2.12 · Elixir 1.20.4 / OTP 29
Page	http://localhost:4000/posts
Files changed	lib/my_app_web/live/post_live/index.ex · lib/my_app/blog_store.ex

How the page is wired

File	Role
lib/my_app_web/router.ex	live "/posts", PostLive.Index — the route
lib/my_app/application.ex	Starts MyAppWeb.BlogStore in the supervision tree (children list)
lib/my_app/blog_store.ex	MyAppWeb.BlogStore — an Agent holding a list of %{id, title, body} maps
lib/my_app_web/live/post_live/index.ex	MyAppWeb.PostLive.Index — mount, events, template

BlogStore plays the role of the **context** from Section 7: the LiveView never touches the posts directly, it calls **list_posts/0**, **create_post/1**, **update_post/2** and, after this lab, **delete_post/1**.

Step	Change	File
1	Fix: validation errors were never shown, so Add Post looked broken	index.ex
2	New store function delete_post/1	blog_store.ex
3	New "delete" event in the LiveView	index.ex
4	Delete button, with a confirm prompt	index.ex (template)
5	Guard: Edit on a post that no longer exists	index.ex
6	The success message hides itself after 3 seconds	index.ex

Step 1 · Fix: “Add Post” seemed to do nothing

A valid post saved fine. But with a title shorter than 3 characters or a body shorter than 10, clicking **Add Post** did nothing: the post was not saved and **no error message appeared**. The server log printed this warning on every keystroke:

SERVER LOG · MIX PHX.SERVER

```
warning: a map with atom keys was given to a form. Maps are always considered
parameters and therefore must have string keys, got:
%{:errors => [{"body", {"should be at least 10 characters", []}},
             {"title", {"should be at least 3 characters", []}}],
  "body" => "short", "title" => "Hi"}
```

There were **two** causes. Both had to be fixed — with only one fixed, still no message appears.

Cause 1 · the errors were put inside the form data

BEFORE (BUG) · LIB/MY_APP_WEB/LIVE/POST_LIVE/INDEX.EX · "VALIDATE" AND "SAVE"

```
errors = validate(params)
form = to_form(Map.put(params, :errors, errors), as: :post)
```

AFTER

```
errors = validate(params)
form = to_form(params, as: :post, errors: errors)
```

to_form/2 treats a map as the form's **data** — the field values. It reads errors only from its **errors:** option.

Map.put(params, :errors, errors) added a fake field called **:errors** (hence the warning about atom keys) and left the form's real error list empty, so the template had nothing to show.

Cause 2 · the error keys were strings

BEFORE (BUG) · VALIDATE/1

```
[{"title", {"can't be blank", []}} | errors]
[{"body", {"should be at least 10 characters", []}} | errors]
```

AFTER · THE SAME FOUR LINES USE ATOMS

```
[{:title, {"can't be blank", []}} | errors]
[{:body, {"should be at least 10 characters", []}} | errors]
```

The template reads the errors with `@form[:title].errors`. Phoenix.HTML keeps only the entries whose key **equals** the field — here the atom `:title` (for `{^field, error} <- errors`). A `"title"` string never matches an atom, so even errors passed the right way would have been filtered out.

Refinement · show an error only once the field has been used

AFTER · TEMPLATE, THE SAME FOR THE BODY FIELD

```
<%= for {msg, _} <- @form[:title].errors, used_input?(@form[:title]) do %>
  <p class="text-sm text-red-600 mt-1">{msg}</p>
<% end %>
```

With **phx-change**, LiveView's JavaScript also sends `_unused_title` / `_unused_body` keys for fields the user has not touched yet. `used_input?/1` returns **false** for those, so typing a title no longer shows “Body can't be blank” at once. In a **for**, an expression after the generator is a **filter**. On submit no `_unused_` keys are sent, so every error shows.

Step 2 · BlogStore.delete_post/1

LIB/MY_APP/BLOG_STORE.EX · NEW FUNCTION, AFTER UPDATE_POST/2

```
def delete_post(id) do
  Agent.get_and_update(__MODULE__, fn posts ->
    case Enum.split_with(posts, &(&1.id == id)) do
      [], _ -> {:error, :not_found}, posts
      [deleted], rest -> {:ok, deleted}, rest
    end
  end)
end
```

- **Agent.get_and_update/2** runs the function **inside the Agent process**. The function returns `{reply, new_state}`: the caller gets **reply**, the Agent keeps **new_state**. Reading and changing happen in one step, so two tabs deleting at once cannot interfere.
- **Enum.split_with/2** splits the list into the posts with this id and all the others.
- No match → reply `{:error, :not_found}` and keep the list as it is. One match → reply `{:ok, deleted}` and keep only the rest.
- Same `{:ok, ...}` / `{:error, ...}` shape as **update_post/2** and the contexts in Section 7.

Step 3 · The "delete" event in the LiveView

LIB/MY_APP_WEB/LIVE/POST_LIVE/INDEX.EX · NEW HANDLE_EVENT/3 CLAUSE

```
# ===== DELETE BUTTON =====
@impl true
def handle_event("delete", %{"id" => id}, socket) do
  id = String.to_integer(id)

  message =
    case BlogStore.delete_post(id) do
      {:ok, _post} -> "Post deleted successfully!"
      {:error, :not_found} -> nil
    end

  socket =
    socket
    |> assign(:posts, BlogStore.list_posts())
    |> show_message(message)

  # If the post being edited was removed, drop back to "Add" mode
  socket =
    if socket.assigns.editing_id == id do
      socket
      |> assign(:editing_id, nil)
      |> assign(:form, to_form(%{"title" => "", "body" => ""}, as: :post))
      |> assign(:page_title, "Posts")
    else
      socket
    end

  {:noreply, socket}
end
```

- **phx-value-id** arrives as a **string** ("8260"); the store's ids are integers, hence **String.to_integer/1**.
- Re-reading **:posts** from the store changes an assign, so LiveView re-renders the table without the deleted row.
- **show_message/2** (Step 6) shows the green banner and starts its timer. For a post that was already gone, **message** is **nil** and no banner shows.
- If the deleted post is the one open in the form (**editing_id**), the form goes back to **Add Post** — otherwise **Update Post** would point at a post that no longer exists.

Step 4 · The Delete button

LIB/MY_APP_WEB/LIVE/POST_LIVE/INDEX.EX · TEMPLATE, ACTIONS CELL OF EACH ROW

```
<td class="px-6 py-4 text-right space-x-4 whitespace-nowrap">
  <button
    phx-click="edit"
    phx-value-id={post.id}
    class="text-indigo-600 hover:text-indigo-900 font-medium text-sm"
  >
    Edit
  </button>
  <button
    phx-click="delete"
    phx-value-id={post.id}
    data-confirm="Are you sure you want to delete this post?"
    class="text-red-600 hover:text-red-900 font-medium text-sm"
  >
    Delete
  </button>
</td>
```

Attribute	What it does
phx-click="delete"	Sends the "delete" event to handle_event/3
phx-value-id={post.id}	Adds %{"id" => "8260"} to the event params
data-confirm="..."	Shows the browser's confirm pop-up first; Cancel sends nothing
space-x-4 whitespace-nowrap	Spaces Edit and Delete apart and keeps them on one line

data-confirm is handled by the **phoenix_html** JavaScript that **assets/js/app.js** imports (**import "phoenix_html"**). Its click listener runs before LiveView's; on **Cancel** it stops the click, so the event never reaches the server.

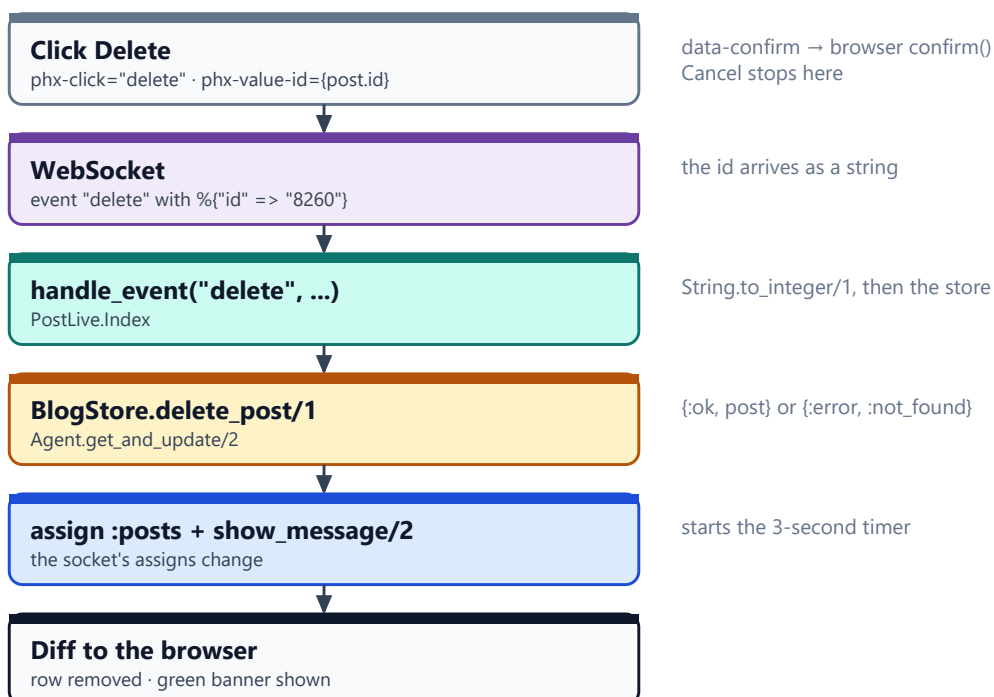


Figure 10. One Delete click. The same LiveView process handles the event, calls the store and sends back only what changed.

Step 5 · Guard: editing a post that is gone

LIB/MY_APP_WEB/LIVE/POST_LIVE/INDEX.EX · THE "EDIT" EVENT

```
def handle_event("edit", %{"id" => id}, socket) do
  id = String.to_integer(id)

  case BlogStore.get_post(id) do
    nil ->
      # Post was removed (e.g. from another tab) - just refresh the list
      {:noreply, assign(socket, :posts, BlogStore.list_posts())}

    post ->
      form = to_form(%{"title" => post.title, "body" => post.body}, as: :post)

      socket =
        socket
        |> assign(:editing_id, id)
        |> assign(:form, form)
        |> assign(:page_title, "Edit Post")
        |> assign(:saved_message, nil)

      {:noreply, socket}
  end
end
```

Each browser tab has its **own** LiveView process with its own copy of **@posts**, but all tabs share **one** BlogStore. Delete a post in tab A and tab B still lists it. Before this guard, clicking **Edit** in tab B called **post.title** on **nil** and crashed that LiveView. Now it simply refreshes the list.

Step 6 · The success message hides itself after 3 seconds

The green banner ("Post created / updated / deleted successfully!") used to stay until the next action. Now the LiveView sets a 3-second timer each time it shows a message. Four small pieces do the job.

6A · TOP OF THE MODULE, AND MOUNT/3

```
# How long a success message stays on screen (ms)
@message_timeout 3_000

# in mount/3, next to the other assigns
|> assign(:saved_message, nil)
|> assign(:message_ref, nil)
```

6B · SHOW_MESSAGE/2 — SHOW A MESSAGE AND START ITS TIMER

```
defp show_message(socket, nil), do: assign(socket, :saved_message, nil)

defp show_message(socket, message) do
  # Tag each timer so an old timer can't hide a newer message early
  ref = make_ref()
  Process.send_after(self(), {:clear_message, ref}, @message_timeout)

  socket
  |> assign(:saved_message, message)
  |> assign(:message_ref, ref)
end
```

6B · THE THREE PLACES THAT SHOW A MESSAGE NOW CALL IT

```
|> show_message(message)           # "save" (create)
|> show_message("Post updated successfully!") # "save" (update)
|> show_message(message)           # "delete"
```

6C · HANDLE_INFO/2 — THE TIMER FIRES

```
# ===== AUTO-HIDE SUCCESS MESSAGE =====
@impl true
def handle_info({:clear_message, ref}, %{assigns: %{message_ref: ref}} = socket) do
  {:noreply, assign(socket, :saved_message, nil)}
end

# Timer from an older message — a newer one is showing, so leave it alone
def handle_info({:clear_message, _ref}, socket), do: {:noreply, socket}
```

6D · TEMPLATE — FADE OUT INSTEAD OF VANISHING

```
<%= if @saved_message do %>
  <div
    id="saved-message"
    phx-remove={
      JS.hide(
        transition: {"transition-opacity duration-500", "opacity-100", "opacity-0"},
        time: 500
      )
    }
    class="mb-6 p-4 bg-green-100 border border-green-300 text-green-800 rounded-lg"
  >
    {@saved_message}
  </div>
<% end %>
```

How it works

Piece	What it does
make_ref/0	Creates a unique value that tags this one message
Process.send_after/3	A LiveView is a process (Figure 4). This asks the BEAM to deliver {:clear_message, ref} to this same process in 3 000 ms. No JavaScript and no sleep: the process keeps handling clicks meanwhile
handle_info/2	LiveView calls it for any plain message a process receives — here, the timer's message
ref twice in one pattern	The first clause binds ref in the message and again in assigns.message_ref. A variable used twice in a pattern must have the same value, so the clause matches only if the timer belongs to the message on screen
Second handle_info clause	Any other (stale) timer is ignored
show_message(socket, nil)	No message (a delete of a post that was gone): no banner, no timer
phx-remove	LiveView runs this JS command when it removes the element: the banner fades from opacity 100 to 0 in 500 ms, then disappears

Why the tag matters: add a post, then delete one 2 seconds later. The first timer fires at 3 s, while the delete message has been on screen for only 1 s. Without the tag it would hide that message early; with it, the old timer is ignored.

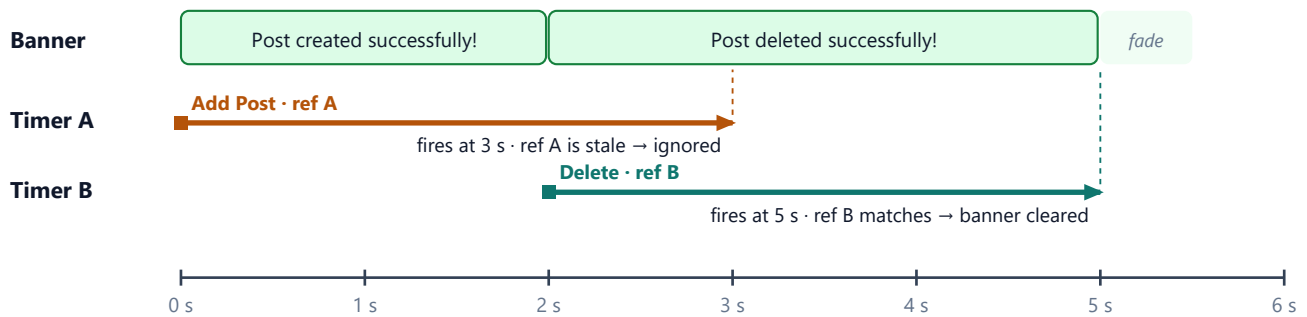


Figure 11. Two messages, two timers. Timer A fires while message B is showing and is ignored; timer B clears its own message 3 s after it appeared. Tested in the browser.

GOOD TO KNOW Typing in the form, **Edit** and **Cancel** still clear the message at once — they set `saved_message` to `nil` directly. To change the delay, edit `@message_timeout`.

Try it

START THE APP

```
cd D:\trainings\cts\session\webapps\my_app
mix compile --warnings-as-errors
mix phx.server
```

#	Do this at http://localhost:4000/posts	Expect
1	Type "Hi" in Title	"should be at least 3 characters" under Title only
2	Add a short body and click Add Post	Both errors show; nothing is saved
3	Fix both fields and click Add Post	The post is listed; the banner fades after ~3 s
4	Click Delete, then Cancel	Nothing happens
5	Click Edit, then Delete on the same row (OK)	Row removed; the form is back to Add Post
6	Add a post, delete it within 2 s	The delete message still stays its full 3 s

LEFT AS AN EXERCISE The update path calls `put_flash(socket, :error, "Post not found")`, but that message never shows: flash messages are drawn by `<Layouts.app flash={@flash}>`, and this template is not wrapped in it (the project's `AGENTS.md` asks for that wrapper). Adding it also adds the standard Phoenix header to the page.

Complete files after the lab

LIB/MY_APP/BLOG_STORE.EX

```
defmodule MyAppWeb.BlogStore do
  use Agent

  def start_link(_opts) do
    Agent.start_link(fn -> [] end, name: __MODULE__)
  end

  def list_posts do
    Agent.get(__MODULE__, &&1)
  end

  def get_post(id) do
    Agent.get(__MODULE__, fn posts ->
      Enum.find(posts, &(&1.id == id))
    end)
  end

  def create_post(attrs) do
    post = %{
      id: System.unique_integer([:positive]),
      title: attrs["title"],
      body: attrs["body"]
    }

    Agent.update(__MODULE__, fn posts -> [post | posts] end)
    {:ok, post}
  end

  def update_post(id, attrs) do
    Agent.get_and_update(__MODULE__, fn posts ->
      case Enum.find_index(posts, &(&1.id == id)) do
        nil ->
          {:error, :not_found}, posts

        index ->
          updated = %{
            id: id,
            title: attrs["title"],
            body: attrs["body"]
          }

          new_posts = List.replace_at(posts, index, updated)
          {:ok, updated}, new_posts
      end
    end)
  end

  def delete_post(id) do
    Agent.get_and_update(__MODULE__, fn posts ->
      case Enum.split_with(posts, &(&1.id == id)) do
        [], _ -> {:error, :not_found}, posts
        [deleted], rest -> {:ok, deleted}, rest
      end
    end)
  end
end
```

```

    end
  end)
end
end

```

LIB/MY_APP_WEB/LIVE/POST_LIVE/INDEX.EX

```

defmodule MyAppWeb.PostLive.Index do
  use MyAppWeb, :live_view

  alias MyAppWeb.BlogStore

  # How long a success message stays on screen (ms)
  @message_timeout 3_000

  @impl true
  def mount(_params, _session, socket) do
    posts = BlogStore.list_posts()

    socket =
      socket
      |> assign(:posts, posts)
      |> assign(:page_title, "Posts")
      |> assign(:form, to_form(%{"title" => "", "body" => ""}, as: :post))
      |> assign(:editing_id, nil)
      |> assign(:saved_message, nil)
      |> assign(:message_ref, nil)

    {:ok, socket}
  end

  # ===== VALIDATE =====
  @impl true
  def handle_event("validate", %{"post" => params}, socket) do
    errors = validate(params)
    form = to_form(params, as: :post, errors: errors)

    {:noreply, assign(socket, form: form, saved_message: nil)}
  end

  # ===== SAVE (Create or Update) =====
  @impl true
  def handle_event("save", %{"post" => params}, socket) do
    errors = validate(params)

    if errors == [] do
      case socket.assigns.editing_id do
        nil ->
          # CREATE
          {:ok, _post} = BlogStore.create_post(params)
          message = "Post created successfully!"

          socket =
            socket
            |> assign(:posts, BlogStore.list_posts())
            |> assign(:form, to_form(%{"title" => "", "body" => ""}, as: :post))

```

```

        |> assign(:editing_id, nil)
        |> show_message(message)

{:noreply, socket}

id ->
  # UPDATE
  case BlogStore.update_post(id, params) do
    {:ok, _post} ->
      socket =
        socket
        |> assign(:posts, BlogStore.list_posts())
        |> assign(:form, to_form(%{"title" => "", "body" => ""}, as: :post))
        |> assign(:editing_id, nil)
        |> show_message("Post updated successfully!")
        |> assign(:page_title, "Posts")

      {:noreply, socket}

    {:error, _} ->
      {:noreply, put_flash(socket, :error, "Post not found")}
  end
end
else
  form = to_form(params, as: :post, errors: errors)
  {:noreply, assign(socket, form: form, saved_message: nil)}
end
end

# ===== EDIT BUTTON =====
@impl true
def handle_event("edit", %{"id" => id}, socket) do
  id = String.to_integer(id)

  case BlogStore.get_post(id) do
    nil ->
      # Post was removed (e.g. from another tab) - just refresh the list
      {:noreply, assign(socket, :posts, BlogStore.list_posts())}

    post ->
      form =
        to_form(
          %{"title" => post.title, "body" => post.body},
          as: :post
        )

      socket =
        socket
        |> assign(:editing_id, id)
        |> assign(:form, form)
        |> assign(:page_title, "Edit Post")
        |> assign(:saved_message, nil)

      {:noreply, socket}
  end
end

```

```

end

# ===== DELETE BUTTON =====
@impl true
def handle_event("delete", %{ "id" => id }, socket) do
  id = String.to_integer(id)

  message =
    case BlogStore.delete_post(id) do
      { :ok, _post } -> "Post deleted successfully!"
      { :error, :not_found } -> nil
    end

  socket =
    socket
    |> assign(:posts, BlogStore.list_posts())
    |> show_message(message)

  # If the post being edited was removed, drop back to "Add" mode
  socket =
    if socket.assigns.editing_id == id do
      socket
      |> assign(:editing_id, nil)
      |> assign(:form, to_form(%{ "title" => "", "body" => "" }, as: :post))
      |> assign(:page_title, "Posts")
    else
      socket
    end

  { :noreply, socket }
end

# ===== CANCEL EDIT =====
@impl true
def handle_event("cancel", _params, socket) do
  socket =
    socket
    |> assign(:editing_id, nil)
    |> assign(:form, to_form(%{ "title" => "", "body" => "" }, as: :post))
    |> assign(:page_title, "Posts")
    |> assign(:saved_message, nil)

  { :noreply, socket }
end

# ===== AUTO-HIDE SUCCESS MESSAGE =====
@impl true
def handle_info({ :clear_message, ref }, %{ assigns: %{ message_ref: ref } } = socket) do
  { :noreply, assign(socket, :saved_message, nil) }
end

# Timer from an older message - a newer one is showing, so leave it alone
def handle_info({ :clear_message, _ref }, socket), do: { :noreply, socket }

defp show_message(socket, nil), do: assign(socket, :saved_message, nil)

```

```

defp show_message(socket, message) do
  # Tag each timer so an old timer can't hide a newer message early
  ref = make_ref()
  Process.send_after(self(), {:clear_message, ref}, @message_timeout)

  socket
  |> assign(:saved_message, message)
  |> assign(:message_ref, ref)
end

# ===== VALIDATION =====
defp validate(params) do
  errors = []

  errors =
    cond do
      String.trim(params["title"] || "") == "" ->
        [{:title, {"can't be blank", []}} | errors]

      String.length(params["title"]) < 3 ->
        [{:title, {"should be at least 3 characters", []}} | errors]

      true ->
        errors
    end

  errors =
    cond do
      String.trim(params["body"] || "") == "" ->
        [{:body, {"can't be blank", []}} | errors]

      String.length(params["body"]) < 10 ->
        [{:body, {"should be at least 10 characters", []}} | errors]

      true ->
        errors
    end

  errors
end

@impl true
def render(assigns) do
  ~H"""
<div class="max-w-4xl mx-auto py-10 px-4">
  <h1 class="text-3xl font-bold mb-2">{@page_title}</h1>
  <p class="text-gray-500 mb-8">No database needed - everything is in memory</p>

  <%= if @saved_message do %>
    <div
      id="saved-message"
      phx-remove={
        JS.hide(
          transition: {"transition-opacity duration-500", "opacity-100", "opacity-0"},

```

```

        time: 500
      )
    }
    class="mb-6 p-4 bg-green-100 border border-green-300 text-green-800 rounded-lg"
  >
    {@saved_message}
  </div>
<% end %>

<!-- ===== FORM ===== -->
<.form
  for={@form}
  id="post-form"
  phx-change="validate"
  phx-submit="save"
  class="bg-white p-6 rounded-xl shadow-md mb-10 space-y-5"
>
  <div>
    <label class="block text-sm font-medium mb-1">Title</label>
    <input
      type="text"
      name="post[title]"
      value={@form[:title].value}
      class="w-full border rounded-lg px-4 py-2 focus:ring-2 focus:ring-indigo-500"
      phx-debounce="300"
    />
    <%= for {msg, _} <- @form[:title].errors, used_input?(@form[:title]) do %>
      <p class="text-sm text-red-600 mt-1">{msg}</p>
    <% end %>
  </div>

  <div>
    <label class="block text-sm font-medium mb-1">Body</label>
    <textarea
      name="post[body]"
      rows="4"
      class="w-full border rounded-lg px-4 py-2 focus:ring-2 focus:ring-indigo-500"
      phx-debounce="300"
    >{@form[:body].value}</textarea>
    <%= for {msg, _} <- @form[:body].errors, used_input?(@form[:body]) do %>
      <p class="text-sm text-red-600 mt-1">{msg}</p>
    <% end %>
  </div>

  <div class="flex gap-3">
    <button
      type="submit"
      phx-disable-with="Saving..."
      class="bg-indigo-600 hover:bg-indigo-700 text-white font-semibold py-2 px-6 rounded-lg"
    >
      {if @editing_id, do: "Update Post", else: "Add Post"}
    </button>

    <%= if @editing_id do %>
      <button

```

```

        type="button"
        phx-click="cancel"
        class="bg-gray-200 hover:bg-gray-300 text-gray-800 font-semibold py-2 px-6 rounded-lg"
      >
        Cancel
      </button>
    <% end %>
  </div>
</.form>

<%!-- ===== TABLE ===== --%>
<div class="bg-white rounded-xl shadow-md overflow-hidden">
  <table class="min-w-full divide-y divide-gray-200">
    <thead class="bg-gray-50">
      <tr>
        <th class="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase">ID</th>
        <th class="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase">Title</th>
        <th class="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase">Body</th>
        <th class="px-6 py-3 text-right text-xs font-medium text-gray-500 uppercase">
          Actions
        </th>
      </tr>
    </thead>
    <tbody class="divide-y divide-gray-200">
      <%= if @posts == [] do %>
        <tr>
          <td colspan="4" class="px-6 py-8 text-center text-gray-400">
            No posts yet. Add one above!
          </td>
        </tr>
      <% else %>
        <%= for post <- @posts do %>
          <tr class="hover:bg-gray-50">
            <td class="px-6 py-4 text-sm text-gray-500">{post.id}</td>
            <td class="px-6 py-4 text-sm font-medium text-gray-900">{post.title}</td>
            <td class="px-6 py-4 text-sm text-gray-600">{post.body}</td>
            <td class="px-6 py-4 text-right space-x-4 whitespace-nowrap">
              <button
                phx-click="edit"
                phx-value-id={post.id}
                class="text-indigo-600 hover:text-indigo-900 font-medium text-sm"
              >
                Edit
              </button>
              <button
                phx-click="delete"
                phx-value-id={post.id}
                data-confirm="Are you sure you want to delete this post?"
                class="text-red-600 hover:text-red-900 font-medium text-sm"
              >
                Delete
              </button>
            </td>
          </tr>
        <% end %>
      </tbody>
    </table>
  </div>

```

```
        <% end %>
      </tbody>
    </table>
  </div>
</div>
"""

end

end
```